Internet věcí je vzrušující obor. Programátor vidí výsledek své práce na fyzických věcech – rozsvítí světlo, změří teplotu, nechá je pokecat si mezi sebou.

Během následujících týdnů pochopíte, jak obyčejný senzor a pár drátů proměnit v plně automatizovaný systém, který lidem šetří čas i peníze, a který ovládají z mobilu.

Čekají Vás tyto moduly v průběhu semestru:

Lokální systém (2 týdny): Oživíte hardware a donutíte ho poslouchat vaše příkazy.

Propojení s PC (3 týdny): Naučíte Arduino mluvit s vaším počítačem.

Vzájemná komunikace (3 týdny): Vaše zařízení si začnou povídat bezdrátově.

Projekt (3 týdny): Vše spojíte dohromady a zjistíte, že umíte více, než když jste se narodili.

Pracujte svým tempem. Když se zaseknete, zeptejte se. Když budete hotoví dřív, pomozte sousedovi. Tohle není závod, ale dobrodružství s týmovou spolupráci na konci.

P01 – Úvod k IoT

Cíle:

Rozumím základním pojmům IoT

Sestavím jednoduché elektronické obvody na nepájivém poli

Rozumím prostředí Arduino IDE

Umím naprogramovat a řídit elektronické obvody pomocí Arduina

Teorie:

Představte si...

...vaše hodinky vědí, že jste dnes uběhli o kilometr víc než včera. Že květináč u okna pošle zprávu "žízním", když zapomenete zalít. Že auto si samo objedná servisní prohlídku dřív, než se stihne porouchat.

Internet věcí je svět, kde fyzické předměty ožívají – měří, přemýšlejí, komunikují a šetří lidem starosti.

IoT zařízení:

Aktivní **IoT zařízení** – embedded systém, který:

Má vlastní identitu (MAC adresa, certifikát)

Aktivně komunikuje po síti (Wi-Fi, ZigBee, mobilní sítě)

Sám od sebe něco dělá – měří, vysílá, reaguje

Příklady: chytrý teploměr, sledovač polohy, chytrá zásuvka

Pasivní prvek – objekt, který není samostatným IoT zařízením, protože aktivně nekomunikuje. Nese pouze informaci:

- QR kód na krabici
- čárový kód na zboží,
- RFID karta bez baterie

Součástí IoT ekosystému se stává až ve chvíli, kdy ho aktivní zařízení přečte a vytvoří se jeho digitální reprezentace.

Pro IoT je zásadní, že zařízení je aktivní – samo posílá data, samo se probouzí, samo rozhoduje.

Embedded system:

Každé moderní zařízení začíná u embedded systému. Jde o věc s vestavěným specializovaným počítačem – pračky, auta, průmyslové stroje nebo senzoru teploty.

Důležité je rozlišovat dvě úrovně:

- Embedded computer je samotná výpočetní jednotka.
- Embedded system je celé funkční zařízení včetně výpočetní jednotky, napájení, senzorů a krytů.

Vytáhněte Arduino. Je to vývojová deska, "stavebnice" pro vývojáře. Má spoustu pinů, USB konektor, převodník napětí, regulátor, diody... Arduino je záměrně univerzální, abyste na něm mohli zkoušet různé věci.
Pokud byste chtěli pomocí Arduina řídit jedoúčelový samozavlažovací chytrý květináč, budete plýtvat zdroji:

- Zbytečně velký
- Stojí víc, než musí
- Spotřebovává energii na věci, které květináč nepotřebuje

Jednoúčelový embedded systém je přesný opak:

- Má minimální paměť a výpočetní výkon pro svůj program
- Má jen ty piny (vstupy a výstupy), které jsou potřebné
- Nemá USB konektor (max. skryté programovací piny)
- Plošný spoj kopíruje tvar zařízení, ve kterém je ukryt

Realita embedded systémů není černobílá. Mezi "čistým" embedded systémem a běžným počítačem existuje celá paleta šedé:

čistý embedded systém – jednoúčelový čip (program zůstává v paměti), minimum hardwaru pro funkci systému, Vhodné k masovému průmyslovému nasazení.

typický embedded systém - arduino, dělá jednu věc, program po vypnutí zůstává v paměti, vhodné pro vývoj, prototypy a malé série.

mezní embedded systém - Raspberry Pi. Uvnitř je počítačová architektura s operačním systémem, ale můžete ho vestavět do robota a chová se jako embedded.

víceúčelové systémy – Průmyslový počítač (IPC) – vypadá jako krabice, uvnitř je PC, ale je navržený pro jediný účel v extrémních podmínkách.

unverzální systémy – Běžný PC. Můžete na něm psát, hrát hry nebo běžet jako server. Není to embedded systém, i když zrovna dělá jen jednu věc.

IoT omezení:

IoT zařízení je embedded systém, který musí splňovat přísná pravidla. Jako vývojáři narazíte na omezení, která na běžném PC nezažijete:

Omezená paměť – programujete pro jednotky kilobajtů RAM, ne pro gigabajty.

Omezená energie – zařízení má často vydržet roky na jednu baterii. 99 % času spí a jen občas se probudí k akci.

Nízká cena – výsledný produkt se musí prodávat za pár dolarů, jinak nemá na trhu šanci.

Vysoká spolehlivost – když se zasekne webová stránka, nikdo neumře. Když se zasekne IoT senzor v chemičce nebo kardiostimulátor ...

Bezpečnost – šifrování na malém čipu s omezeným výkonem není triviální. A přitom posíláte citlivá data.

Komunikace takových zařízení se neobejde bez speciálních protokolů. HTTP je pro ně příliš "ukecané a žravé". Místo něj se používají minimalistické jazyky jako MQTT nebo CoAP, které pošlou jen holá data a šetří každý bajt.

Na co se ptejte při Vašem samostudiu?

Co je věcí a co zařízením v IoT?

Jaký je rozdíl mezi digitalizací, digitální reprezentací a digitálním dvojčetem?

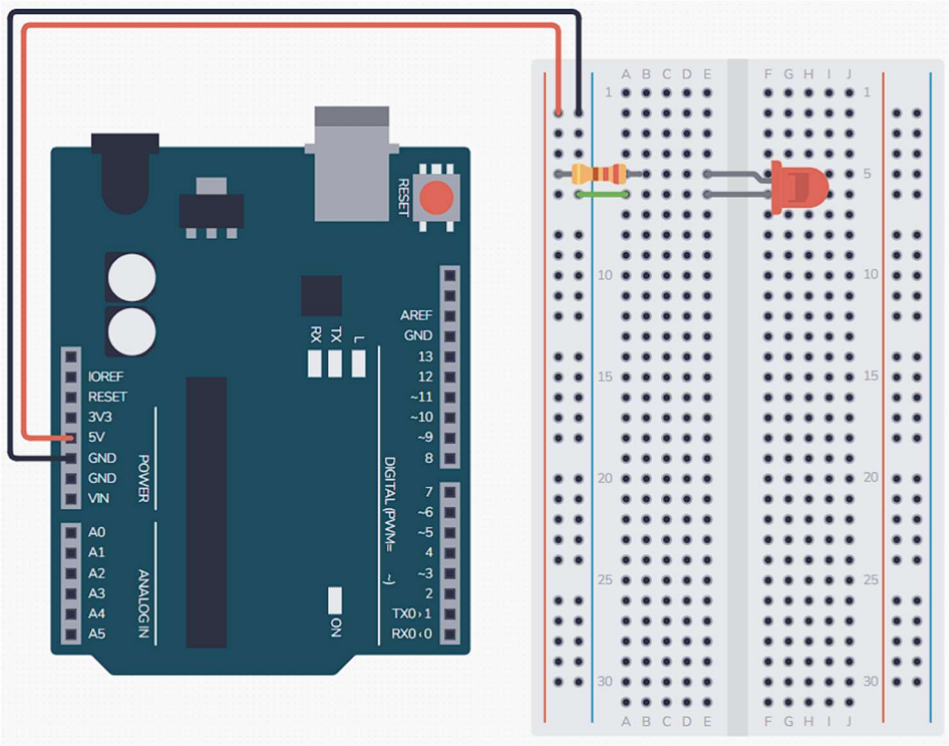
Co je senzor a co je aktor?

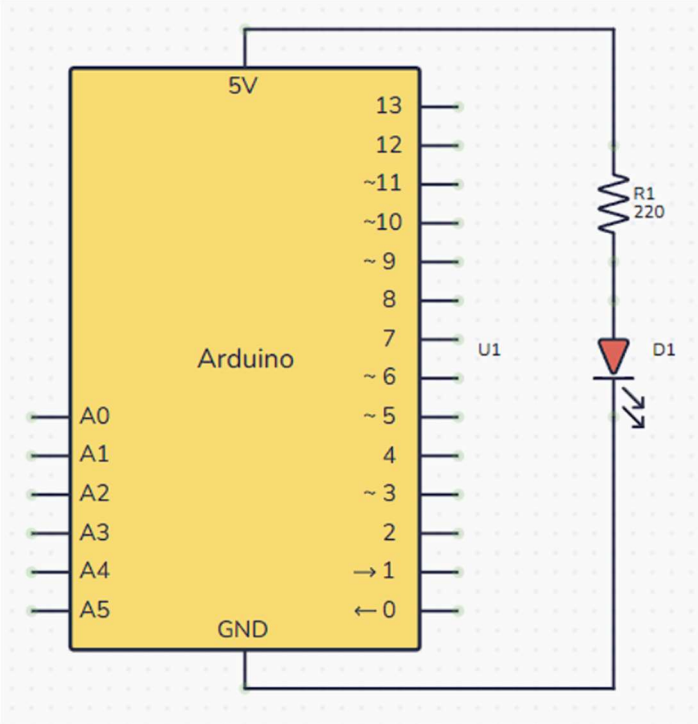
Co znamená firmware a čím se liší od běžného softwaru?

Které komunikační technologie se v IoT používají a jaké mají výhody a nevýhody?

Příklad 1 – jednoduchý elektronický obvod

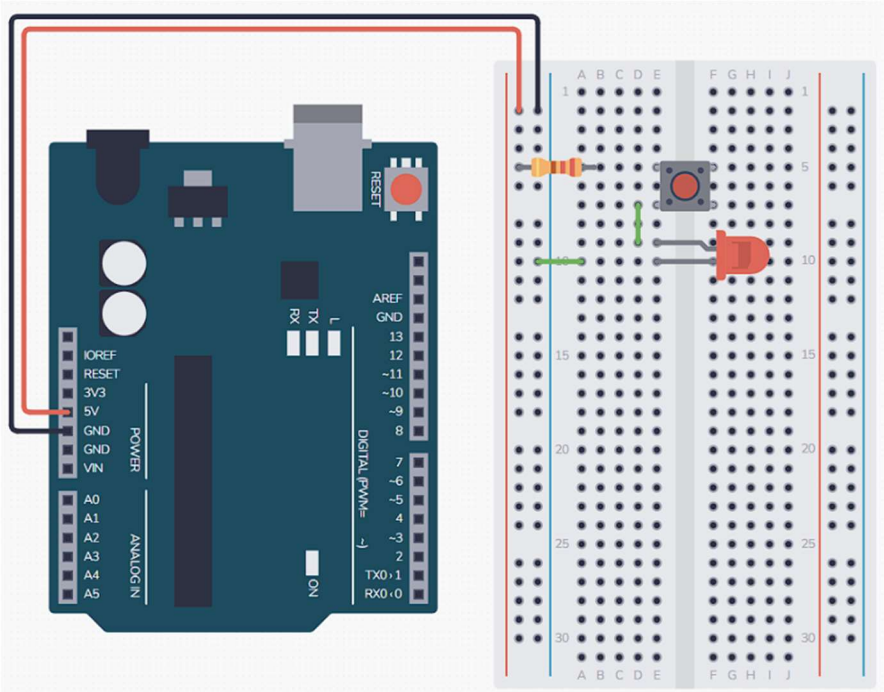
Před zapojením součástek vždy vypoj USB a napájení Arduina! Zapoj elektronický obvod a poté zapni napájení. Na LED se dívej shora.

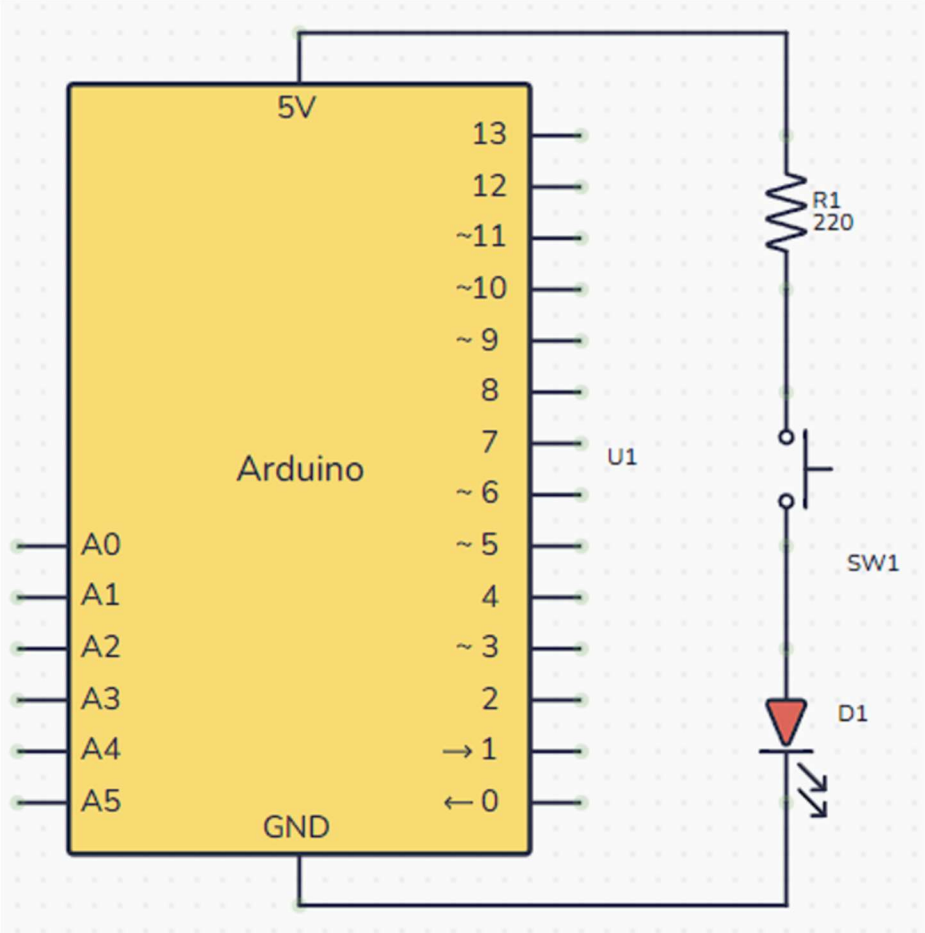




Máte obyčejný elektronický obvod. Arduino zde slouží jako zdroj, ne jako řízení.
Tohle se dá počítat maximálně jako IoT pasivní prvek (součást IoT by z něj musel udělat nějaký externí senzor, který by snímal LED a zapisoval do systému jeho stavy).

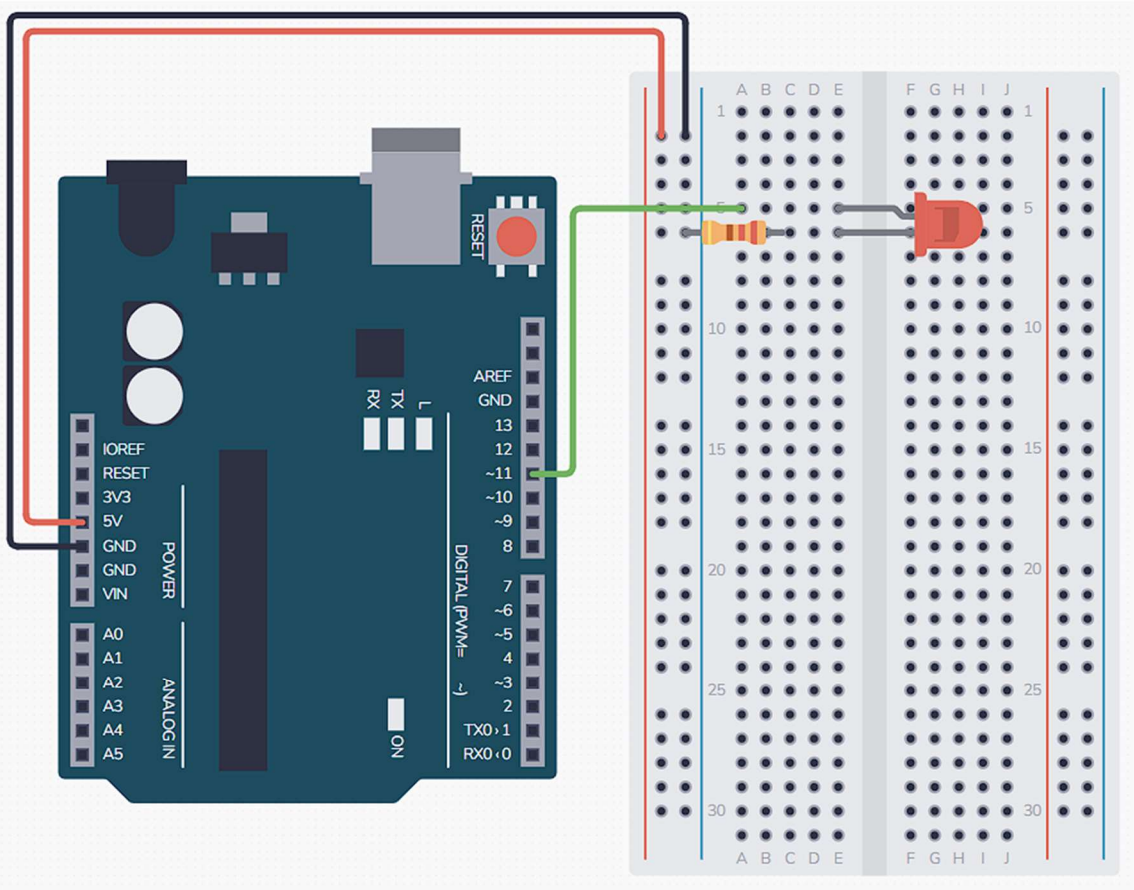
Příklad 2 – elektronický obvod ovládaný tlačítkem

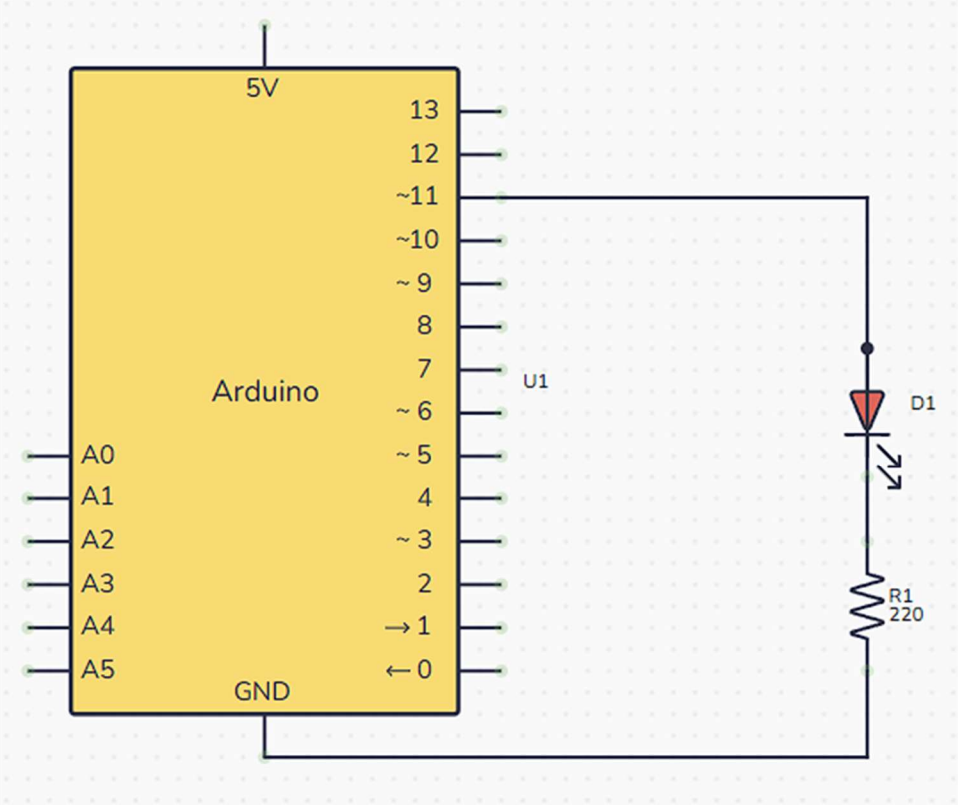




Opět jen elektronický obvod s možností ovládání přes tlačítko. Vypnul jsi napájení před zapojením? Svítí LED pokud tlačítko držíš?

Příklad 3 – jednoduchý embedded systém





Pro řízení přes Arduino je nutné jej naprogramovat.

```
// Blikání LED na pinu 11
// LED je zapojena mezi pin 11 a GND (přes rezistor 220 Ω)

const int ledPin = 11;      // pin, ke kterému je připojena LED

void setup() {
  pinMode(ledPin, OUTPUT);  // nastavíme pin jako výstupní
}

void loop() {
  digitalWrite(ledPin, HIGH); // rozsvítíme LED (výstup HIGH = 5V)
  delay(1000);                 // počkáme 1 sekundu
  digitalWrite(ledPin, LOW);  // zhasneme LED (výstup LOW = 0V)
  delay(1000);                 // počkáme 1 sekundu
}
```

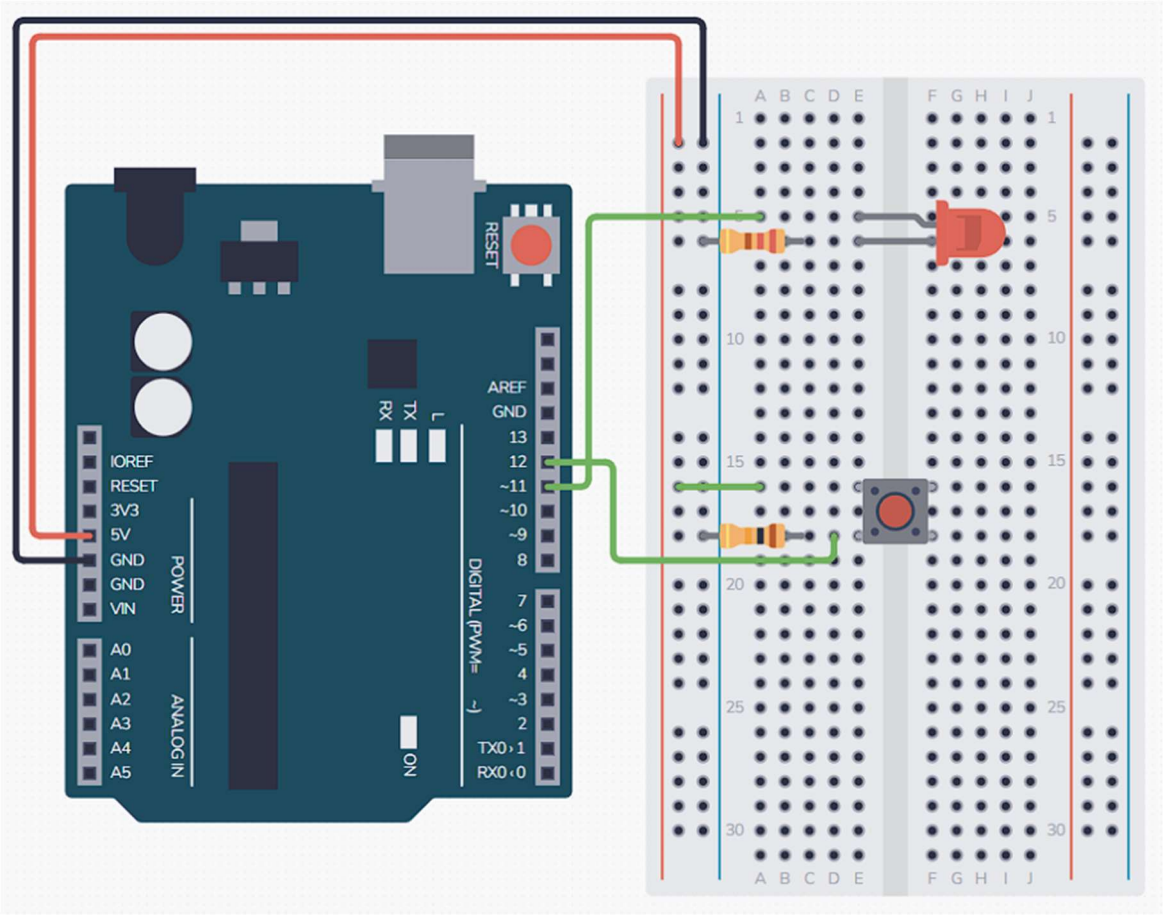
Otázky k zamyšlení:

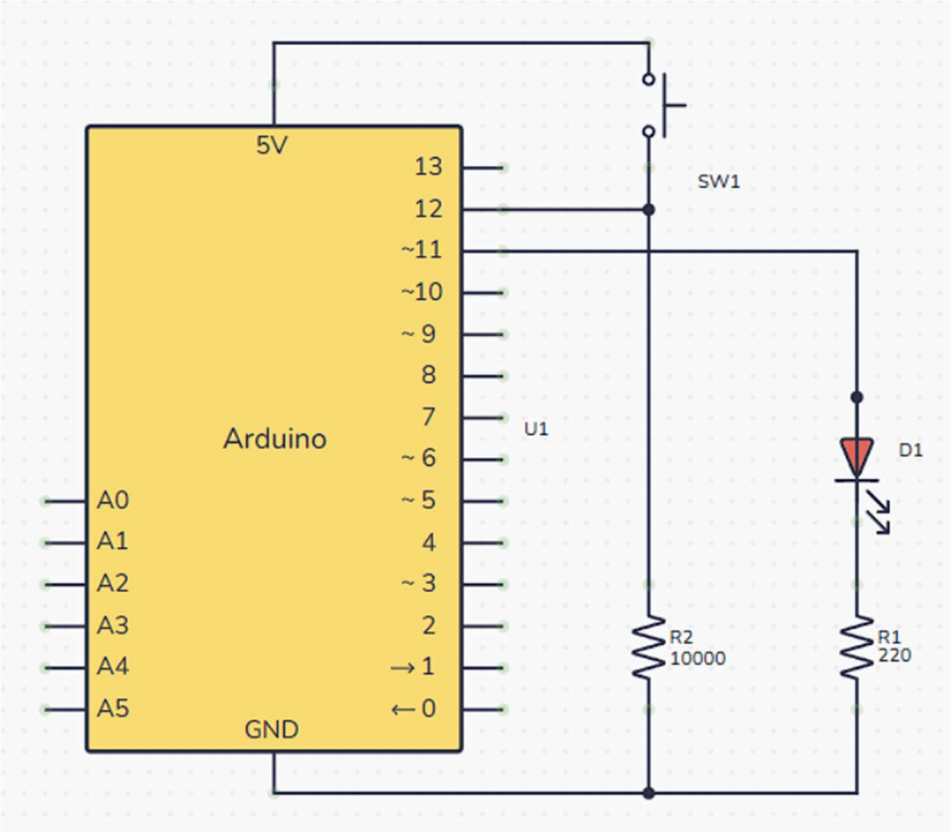
- Co se stane, když změníte hodnotu v delay na 200? A na 5000?
- Která část kódu zajišťuje, že blikání nikdy neskončí?

Tohle je už jednoduchý embedded systém. Vyzkoušej si následující experiment. Vypni napájení a znovu zapni. Arduino by si měl uchovat svůj program, firmware, jde o embedded systém. Totéž neplatí pro PC, nebo program přes který konfigurujete arduino.

Proč ještě nemůžeme mluvit o IoT zařízení, ale je to jen embedded systém? Co nám chybí aby se z našeho embedded systému stalo IoT zařízení?

Příklad 4 – ovládaný embedded systém





```
// Ovládání LED na pinu 11 pomocí tlačítka na pinu 12
// Tlačítko je zapojeno mezi pin 12 a VCC, s pull-down rezistorem 10 kΩ
k GND
// LED je na pinu 11 (s rezistorem 220 Ω)

const int ledPin = 11;       // pin s LED
const int buttonPin = 12;     // pin s tlačítkem

void setup() {
  pinMode(ledPin, OUTPUT);    // LED je výstup
  pinMode(buttonPin, INPUT);   // tlačítko je vstup (čteme napětí)
}

void loop() {
  int buttonState = digitalRead(buttonPin); // přečteme stav tlačítka
(HIGH = stisknuto, LOW = puštěno)

  if (buttonState == HIGH) {   // pokud je tlačítko stisknuté
    digitalWrite(ledPin, HIGH); // rozsvítíme LED
  } else {                      // pokud není stisknuté
    digitalWrite(ledPin, LOW);  // zhasneme LED
  }
}
```

Otázky k zamyšlení:

- **Proč musíme použít digitalRead? Co by se stalo, kdybychom napsali jen if (buttonPin == HIGH)?**
- **Jak byste program změnili, aby se LED rozsvítila při prvním stisku a zhasla při druhém (přepínání)?**

Sada úloh pro ty, kteří si chtějí procvičit programování. Využijte stejné zapojení elektroniky jako u příkladu 4. Budete měnit jen firmware embedded systému.

Úloha	Chování	Co si procvičíš?
1. Držení svítí	LED svítí, dokud držíš tlačítko	Základní podmínka, čtení v reálném čase
2. Přepínání (toggle)	První stisk rozsvítí, druhý zhasne	Práce se stavovou proměnnou, ukládání minulého stavu
3. Časované zhasnutí	Stisk rozsvítí LED na 3 vteřiny, pak sama zhasne	Časovač bez blokování (millis()), návrat k základům
4. Blikání 3×	Stisk = LED 3× zabliká a pak zhasne	Cyklus for, sekvence operací
5. Počítadlo stisků	Každý 5. stisk rozsvítí LED na 1 vteřinu	Počítání, modulo operátor
6. Morseovka	Krátký stisk = tečka, dlouhý = čára. Po pauze se přehraje	Detekce délky stisku, stavový stroj (FSM)
7. Postupné blikání	První stisk = 1× blikni, druhý = 2×, třetí = 3×, pak zase od začátku	Řízení složitější logiky, reset stavu
8. Vymysli vlastní chování	Stisk = SOS Tlumené osvětlení přes PWM	

Hrejte si, zkoumejte a za týden se podíváme na elektronickou část IoT podrobněji a naprogramujeme už něco složitějšího.